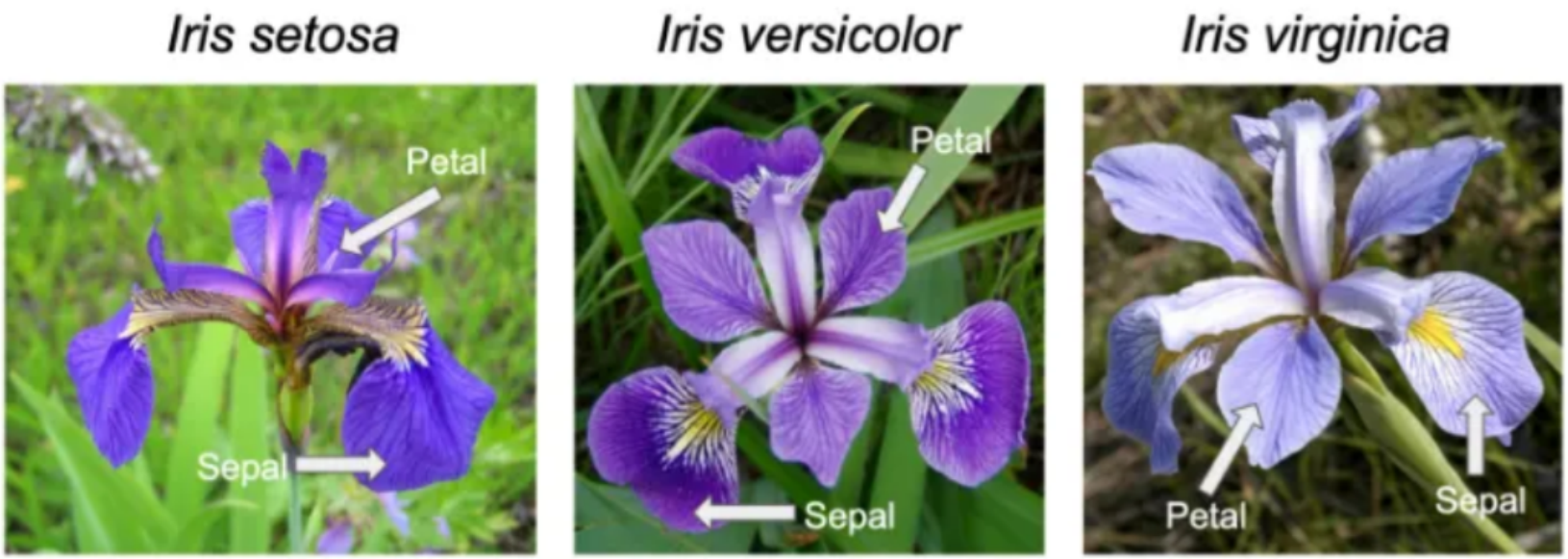
Decision Trees

Albert Y. Kim

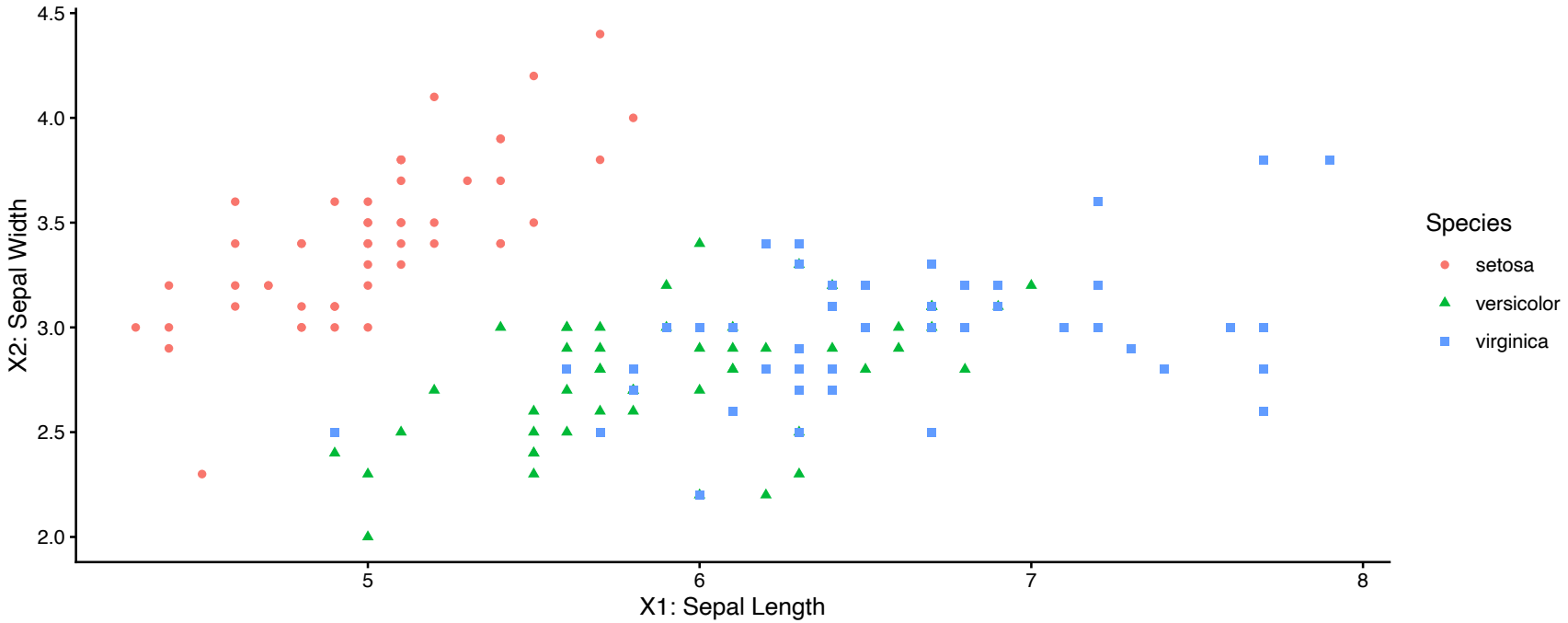
Introduction

The famous statistician Ronald Fisher collected 50 samples from each of three species of Iris. He measured the dimensions of the sepal and petal of each sample. Image source



Introduction

Shown below are the sepal dimensions of these flowers. Can sepal dimensions be used to accurately classify y: the species of iris?



Classification with k-nearest neighbors

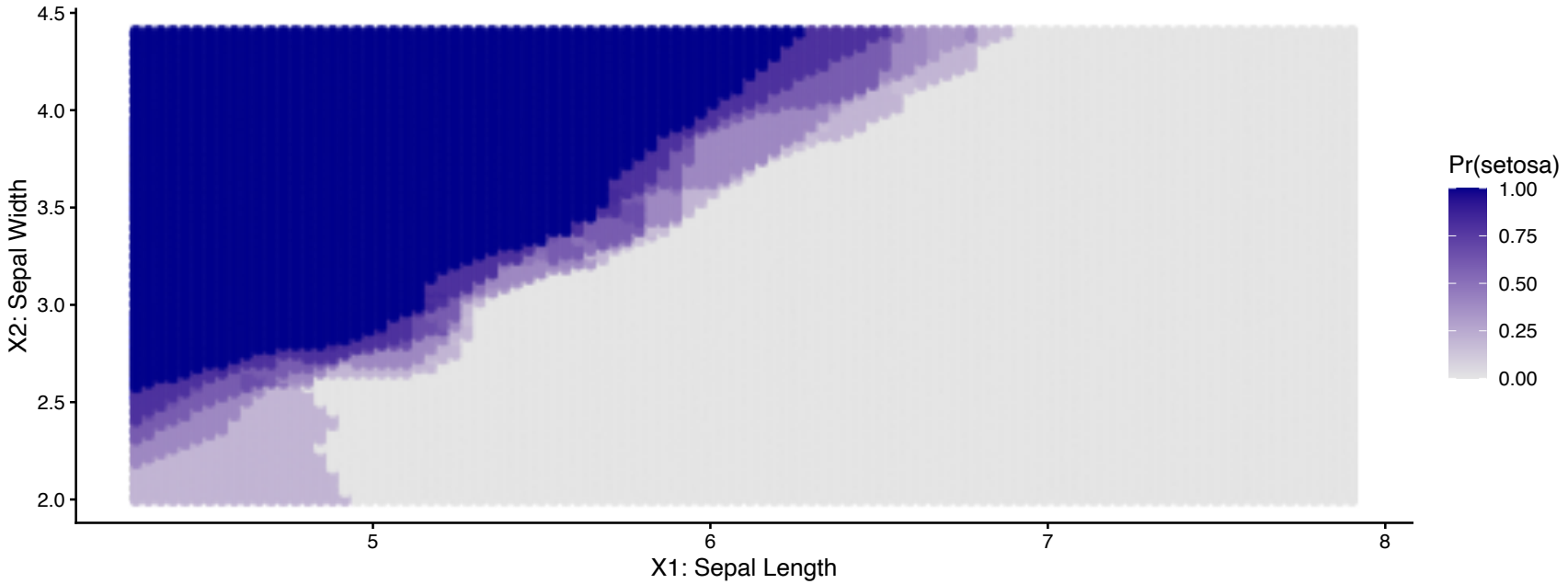
Let's run k-Nearest Neighbors with $k = 5$. Thus for each (X1, X2) combination in the 2D grid, we can compute the Probability of

1. Setosa (red circles)
2. Versicolor (green triangles)
3. Virginica (blue squares)

where the probabilities only take on 1 of 6 values: $\frac{0}{5}, \frac{1}{5}, \frac{2}{5}, \frac{3}{5}, \frac{4}{5}, \frac{5}{5}$

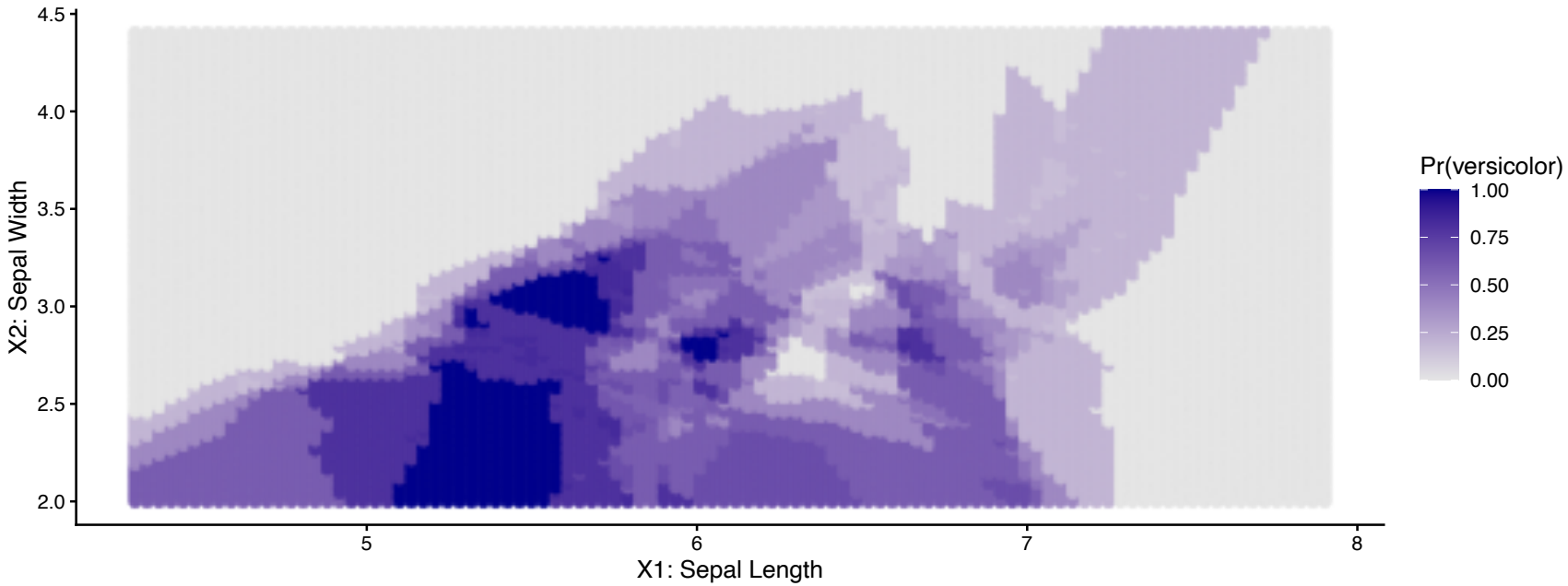
Probability of Setosa using knn

Classification boundary is somewhat clean here. . .



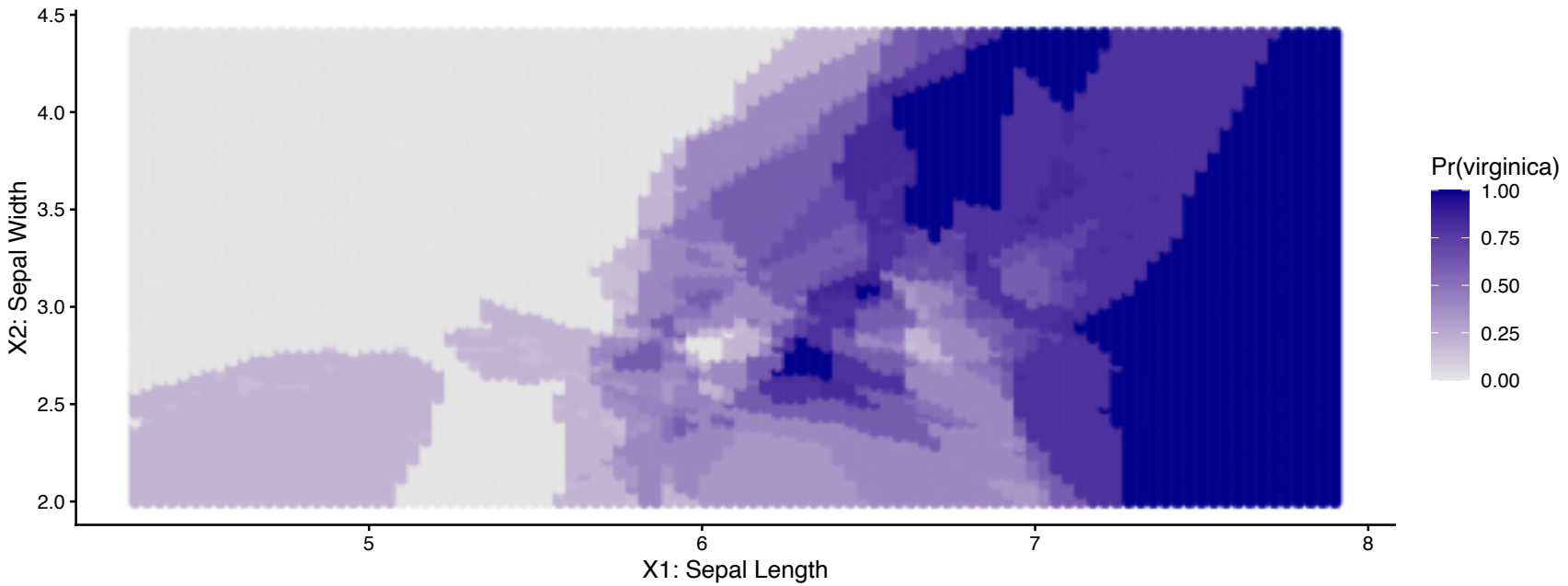
Probability of Versicolor using knn

but highly irregular and seem somewhat haphazard here. . .



Probability of Virginica using knn

same here. . .



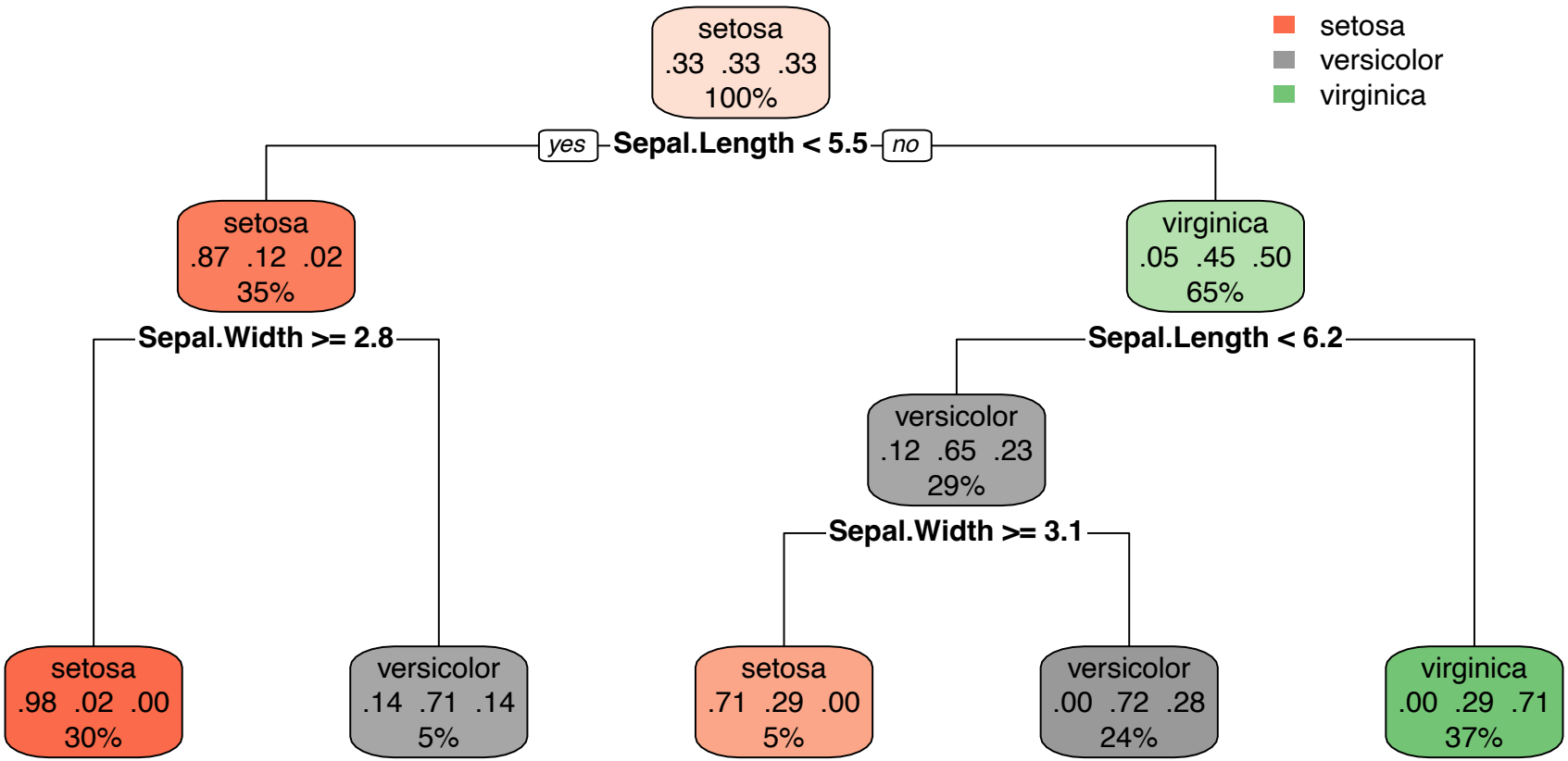
Decision trees

Decision trees aka Classification and Regression Trees (CART) are trained by recursively partitioning the p-dimensional space defined by the explanatory variables i.e. by making binary splits

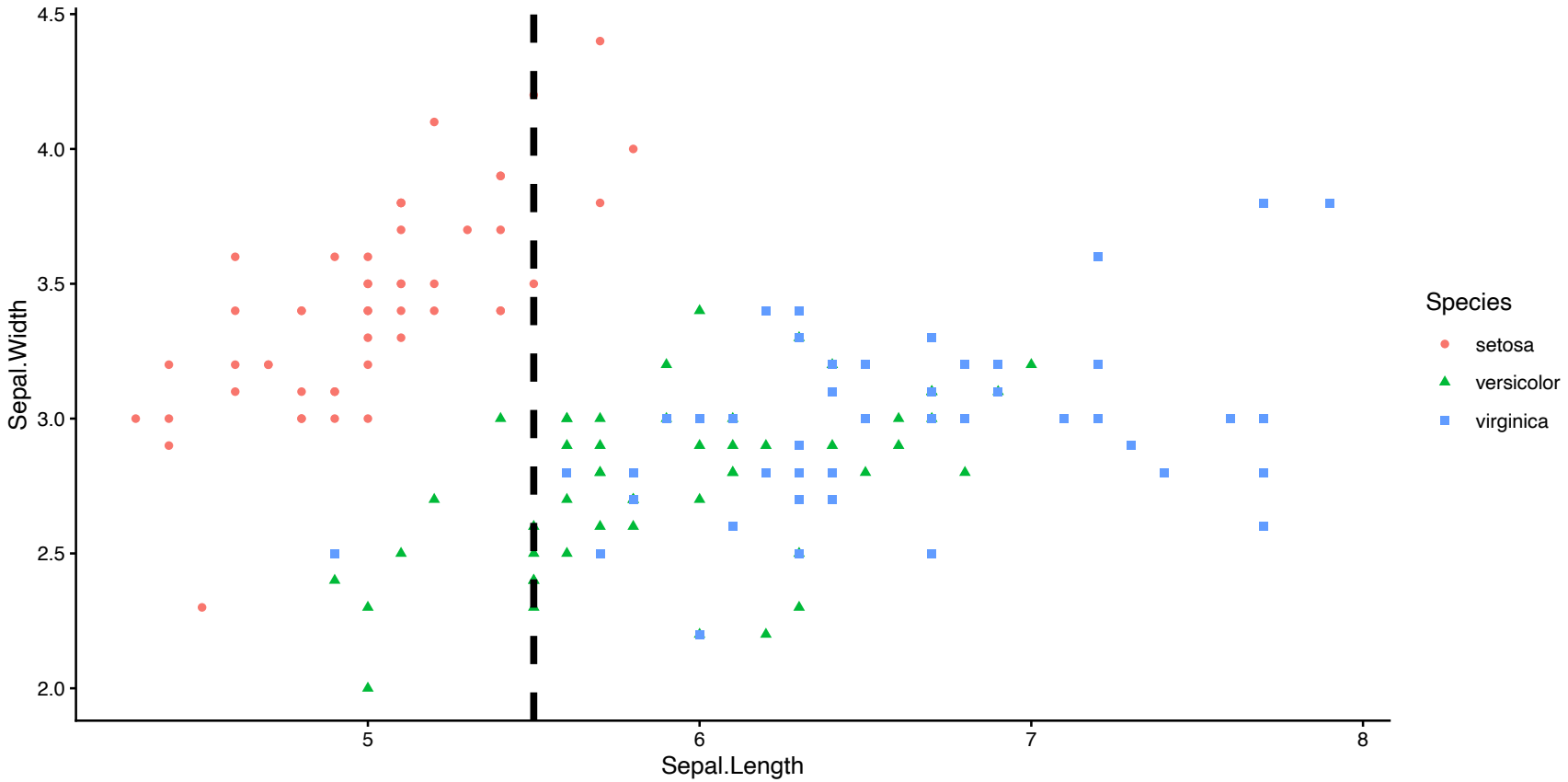
1) Starting with a "parent" node, search for a splitting rule that maximizes the *homogeneity* or *purity* of the "child" nodes
2) Next, considering each node that hasn't yet been split, find another splitting rule that maximizes *purity*
3) Repeat until a stopping criteria has been reached

We'll soon discuss how splitting rules are determined and how the algorithm terminates, but we'll first see what this looks like for the Iris data.

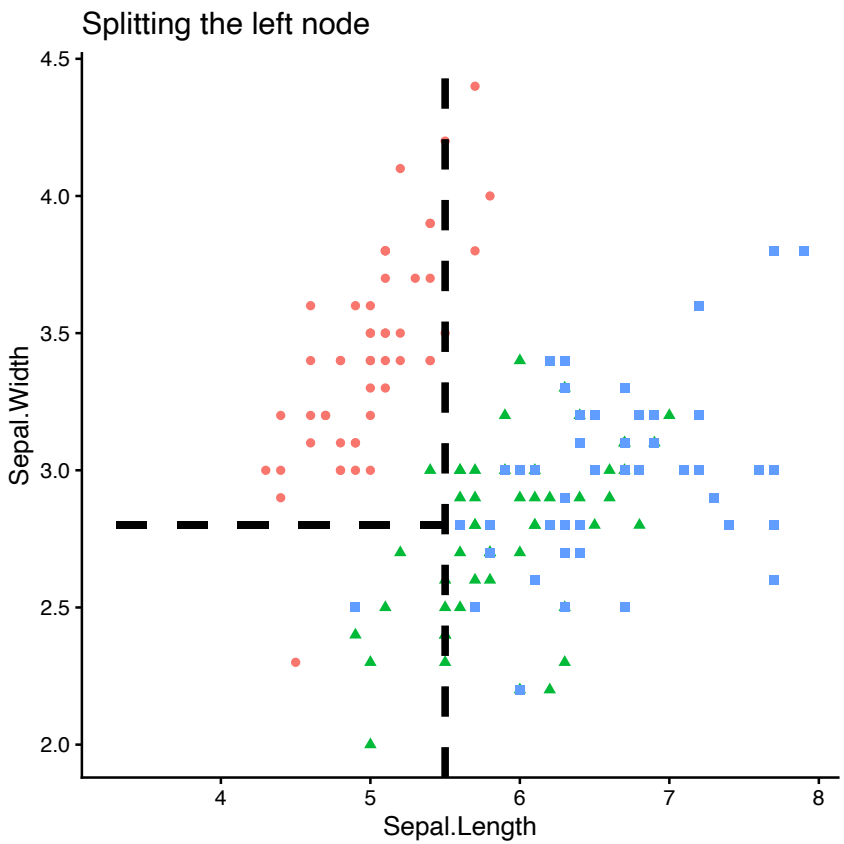
Example: decision tree for iris data

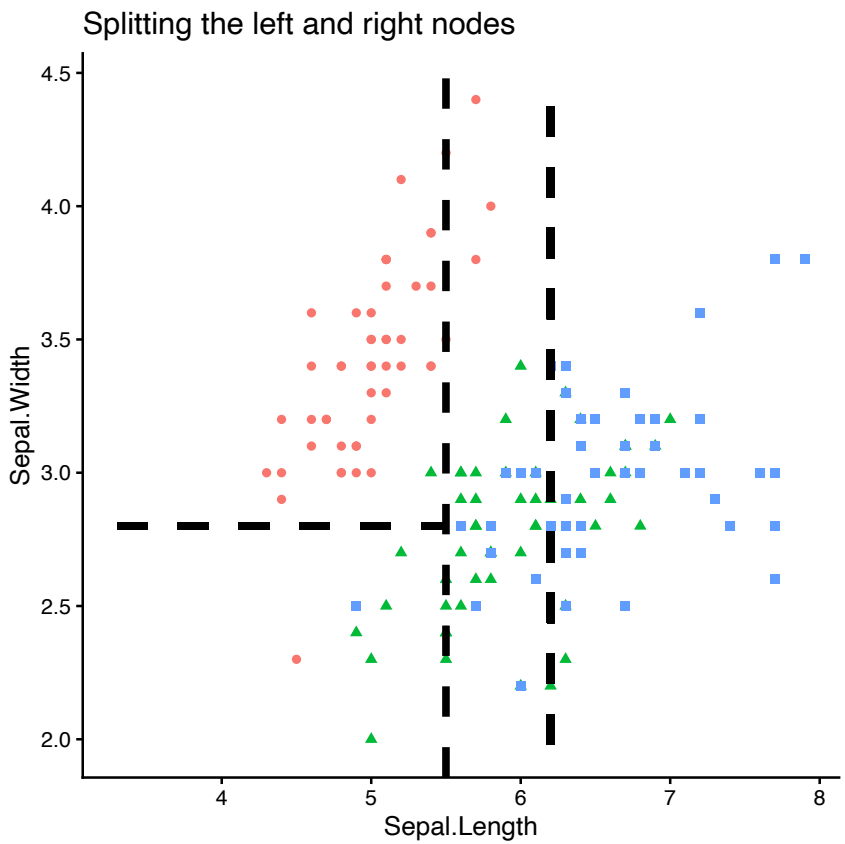


Example (first split)

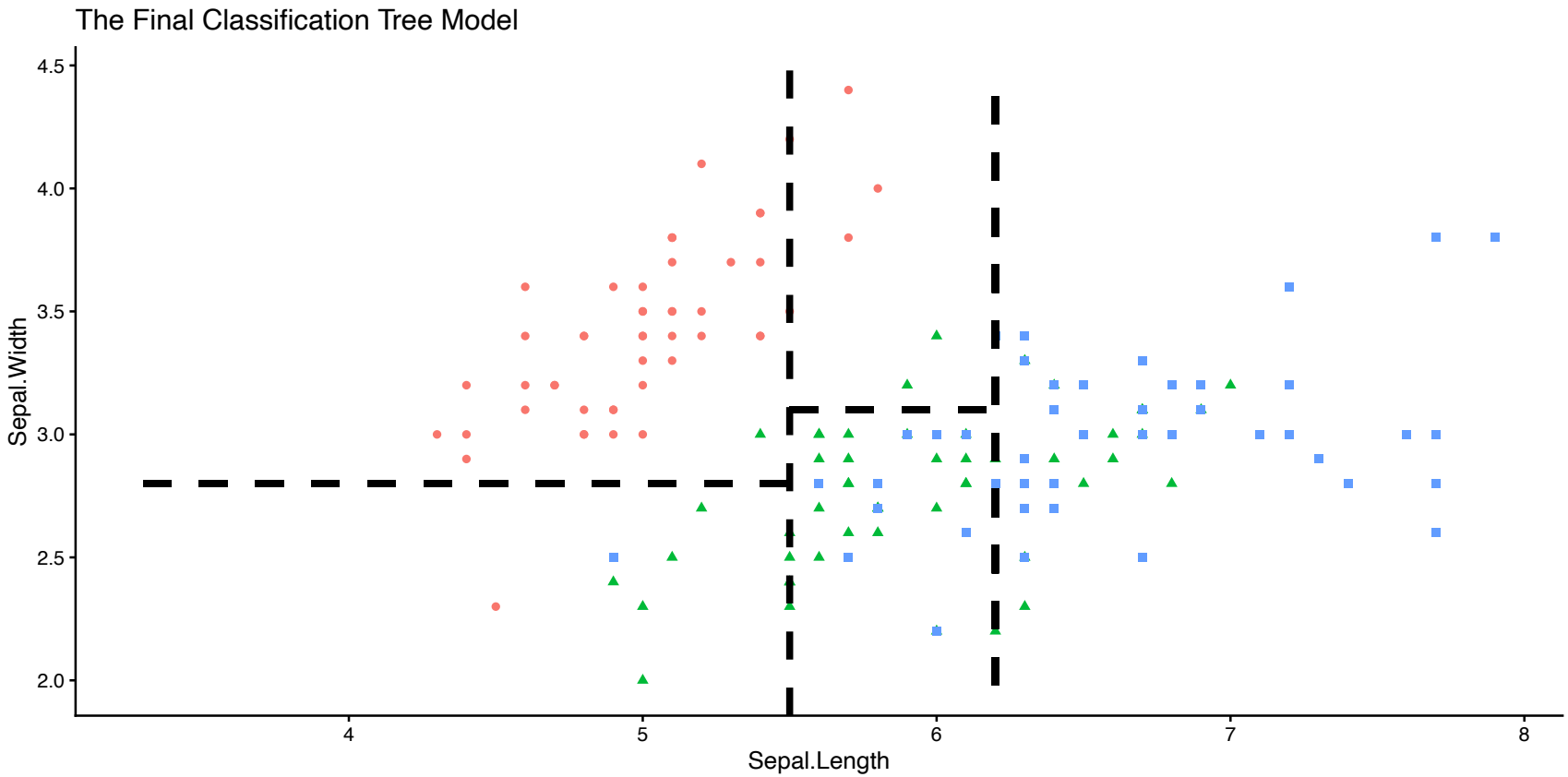


Example (second split)

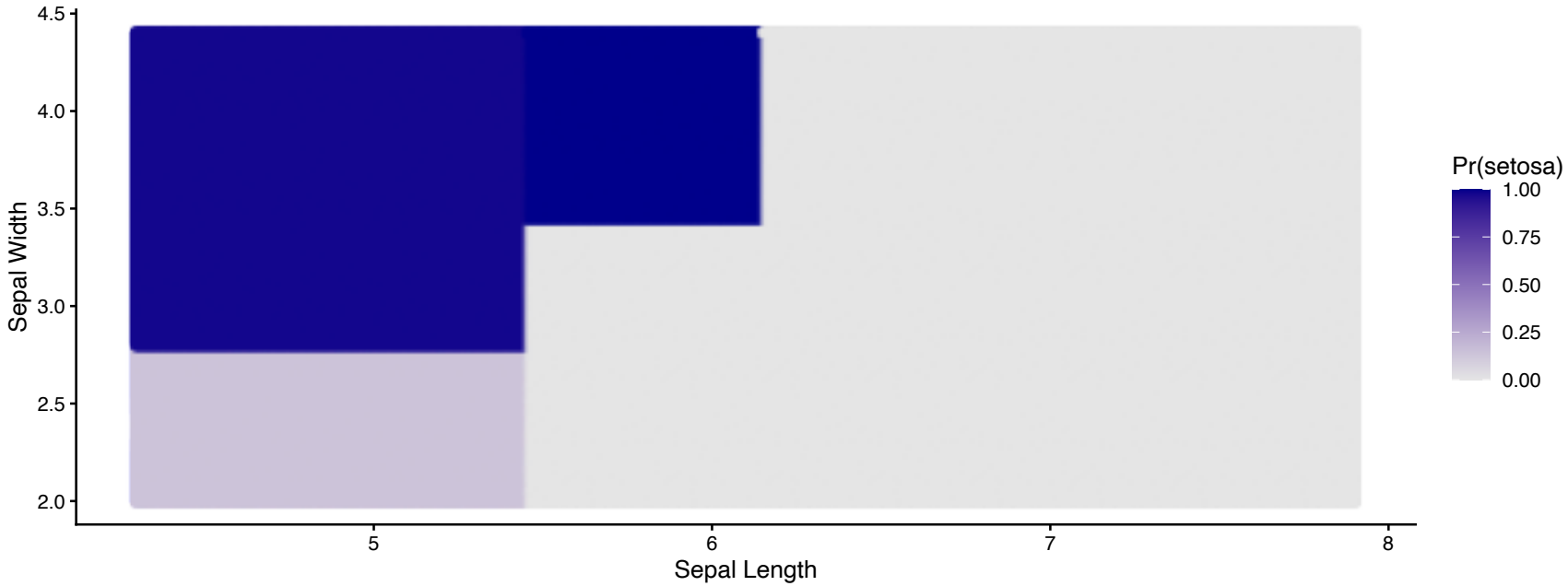




Example (final model)

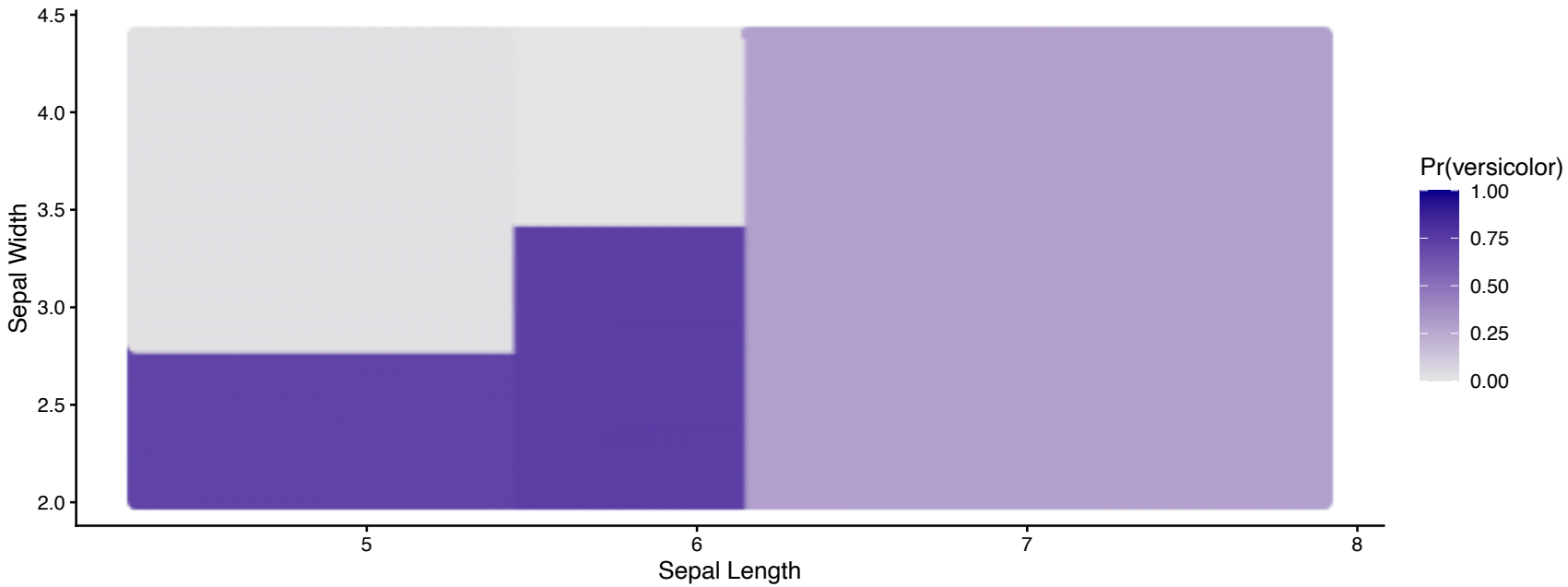


Probability of Setosa using Decision Tree



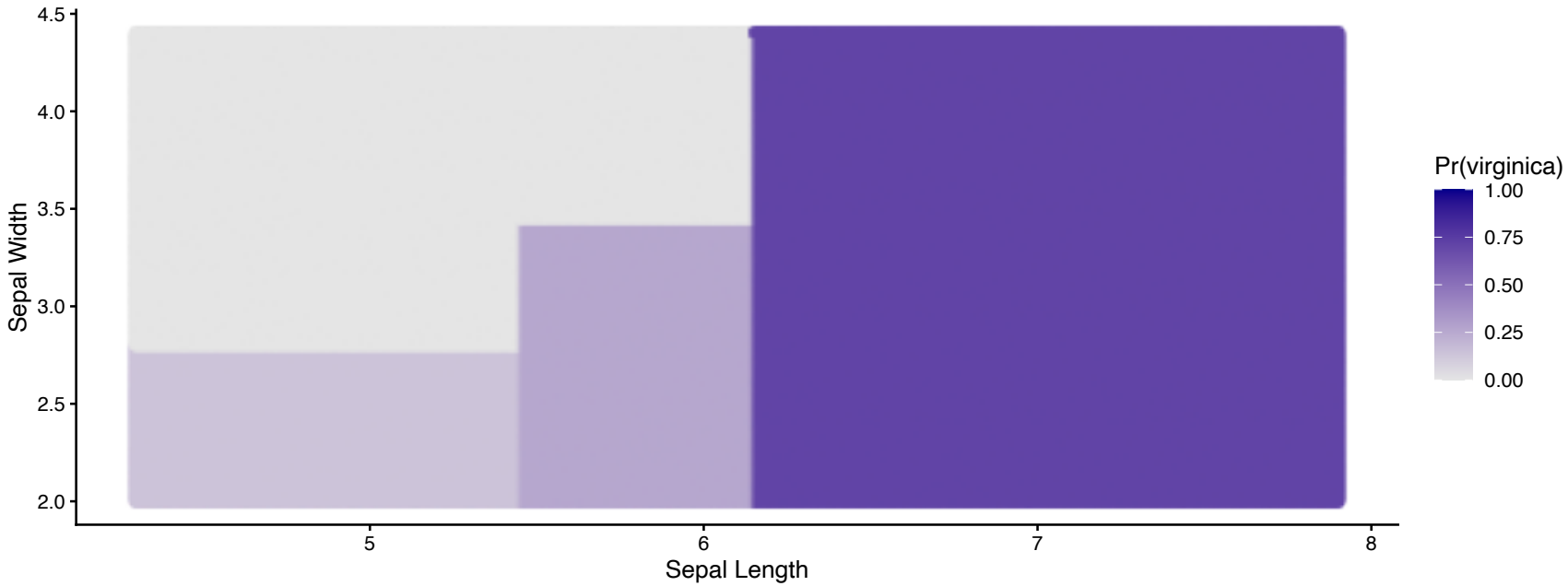
Probability of Versicolor using Decision Tree

but highly irregular and seem somewhat haphazard here. . .



Probability of Virginica using Decision Tree

same here. . .



Splitting criteria: Gini Impurity

Decision trees must learn their splits using an objective criteria, the most common criteria is *Gini impurity* where k is the number of levels/classes in y:

$$Gini = \sum_{j=1}^{k} p_j(1-p_j) = 1 - \sum_{j=1}^{k} p_j^2$$

- For binary classification, this reduces to $p_1(1-p_1) + p_2(1-p_2)$
- Other splitting criteria exist; however, Gini impurity is the default in `sklearn`

Gini Gain

Gini impurity gives us a measure of the impurity of a tree at given depth. The best split is the one that produces the largest decrease (or *Gini gain*) in the resulting child nodes.

In our example tree:

Decision Tree Learning

Building a tree involves iterating between two steps:

1. Finding the optimal splitting point within each variable
2. Selecting the variable to split on

Even if an exhaustive approach were taken, this can be done fairly quickly as there are $n-1$ possible split points per variable and p variables

Regression tasks (numerical outcomes)

Applying the decision tree algorithm to data with a numeric outcome involves the following changes:

- Predicted outcomes are the average outcome $\overline{y}$ in a node
- Mean squared error is used instead of Gini impurity as a measure of impurity

Other splitting criteria, such as mean absolute error or Poisson deviance are available in `sklearn`

Stopping the algorithm

Decision trees can be grown until every terminal node is perfectly pure; however, such trees will be very overfit to the training data. We can control this by:

1. Restricting the maximum depth of the tree (ie: the number of sequential rules)
2. Allowing only nodes of sufficient size be eligible for splitting
3. Requiring a certain improvement in purity for a split to occur

Generally, maximum depth is the most important factor to consider as it directly relates to the complexity of a tree

What we've learned so far

We've now introduced two "classic" supervised machine learning algorithms:

- KNN
  - very flexible, but produces irregular/inconsistent classification boundaries
  - sensitive to re-scaling of x variables since distances are involved
  - computationally inefficient with large data sets
- Decision trees
  - structured, but can still (eventually) learn non-linear patterns
  - more easily interpretable
  - fairly robust to outliers and missing values
  - prone to overfitting at higher max depths

KNN vs Decision Trees

Cross-validation provides an objective strategy for choosing between these models, but we should be aware of their strengths/weaknesses.